FileDrop v1.0

Large File Transfer System

LAN · Internet · Up to 100 GB

Project Name
FileDrop — Peer-to-peer file transfer

Version
2.0.0 (April 2026)

Language / Runtime
Python 3.6+ (standard library only)

Prepared by
Claude · Anthropic

Max File Size
100 GB per file (no hard limit)

Transfer Modes
LAN (local network) + Internet (ngrok/tunnel)

Interface
Web browser — sender UI + receiver UI

Dependencies
None — zero pip installs required

Submitted by - Chandashekhar Tyagi

chandershekhar8433@gmail.com

+918439443923

This document is a comprehensive technical report covering system architecture, data flow, API reference, configuration, and future roadmap for FileDrop v1.0

1. Executive Summary

FileDrop is a self-contained, zero-dependency Python application that allows any two computers — whether on the same office network or on opposite sides of the world — to transfer files of up to 100 GB directly between them. It runs entirely in the background on the receiving machine and exposes a clean, modern web interface that both parties open in an ordinary browser. There is nothing to install on the sender's machine, no account to create, and no file size cap imposed by a cloud provider.

The system solves a real and common problem: standard file-sharing methods all break down at large file sizes. Email attachments are capped at 25 MB. Cloud services like Google Drive and Dropbox throttle upload speed and impose storage quotas. USB drives require physical presence. FTP clients require technical setup. FileDrop has none of these limitations.

The project is built on three core ideas:

- **Chunked transfer:** Files are never loaded into memory all at once. Instead, they are sliced into 4 MB pieces that are uploaded sequentially, allowing stable transfers of arbitrarily large files over any connection.
- **Dual-mode networking:** On a local network, the receiver's LAN IP address is used directly for maximum speed (up to ~100 MB/s on Gigabit Ethernet). For internet transfers, a free tunnelling tool (ngrok) creates a secure public URL that forwards traffic to the local server.
- **Browser-native UI:** Both sender and receiver interact through purpose-built web pages served directly by the Python process. No native app, no Electron wrapper — just HTML, CSS, and JavaScript over HTTP.

Metric	Value	Metric	Value
Max file size	100 GB	**Chunk size**	4 MB
Python version	3.6+ (no pip)	**Platforms**	Windows, macOS, Linux

2. Background and Motivation

2.1 The Problem

Transferring large files between computers is surprisingly difficult when standard tools are unavailable or impractical. Consider the following everyday scenarios:

- **A video editor** needs to send a 40 GB render to a client. WeTransfer's free tier caps at 2 GB. Google Drive is full. The client doesn't have Dropbox.
- **A developer** needs to move a 15 GB virtual machine image from a work laptop to a home machine. Emailing is impossible. A USB drive is not at hand.
- **A researcher** needs to share 80 GB of raw experimental data with a collaborator in another city. Cloud storage is too slow and the institution's VPN blocks large transfers.

In each case, the person needs a simple, fast, direct transfer tool that works without accounts, configuration, or size limits. FileDrop is exactly that tool.

2.2 Why Not Use Existing Tools?

Tool	Size Limit	Requires Account?	Speed on LAN
Email (Gmail)	25 MB	Yes	N/A
Google Drive	15 GB (free)	Yes	~10–30 MB/s
WeTransfer (free)	2 GB	No	~5–15 MB/s
USB drive	Drive capacity	No	~100 MB/s
FTP server	Unlimited	Requires setup	~100 MB/s
FileDrop	**100 GB+**	**No**	**~100 MB/s**

2.3 Design Goals

FileDrop was designed with five non-negotiable principles:

1. **Reliability:** A 100 GB transfer must succeed even on a slow or intermittent connection. It must never crash the browser or exhaust server memory.
2. **Zero setup for the sender:** The person sending the file needs only a web browser. They should not need to install anything.
3. **Real-time visibility:** Both parties should be able to see live progress — bytes transferred, speed in MB/s, and estimated time remaining.
4. **Clean code separation:** Python logic, HTML structure, CSS styling, and JavaScript behaviour should each live in dedicated files. No mixed-language spaghetti.
5. **Extensibility:** The codebase should be easy to extend with authentication, checksums, and parallel uploads in the future without rewriting core logic.

3. System Architecture

3.1 High-Level Overview

FileDrop is a single-process Python HTTP server. It serves two web pages (sender and receiver) and exposes a small REST-like API for the transfer protocol. The entire codebase is split into six Python modules and two sets of static assets (HTML, CSS, JavaScript), each with a single clearly defined responsibility.

> **Architecture Principle**
> Each file in the project answers exactly one question. server.py answers 'How do I start?' config.py answers 'What are the settings?' handler.py answers 'How do I route this request?' This makes the codebase navigable by anyone reading a single file — they never need to scroll through 1,000 lines to understand one concept.

3.2 Module Breakdown

The table below describes every file in the project, what it is responsible for, and why it is a separate file rather than part of a larger module.

File	Type	Responsibility & Explanation
`server.py`	Python	Entry point. The only file the user runs. Parses CLI arguments (--port, --dir, --host), applies them to Config, creates the upload directory, detects the LAN IP, prints the startup banner, and starts the HTTPServer. Kept minimal so any developer can understand startup in under 30 seconds.
`config.py`	Python	Centralised settings class. All tuneable constants live here: CHUNK_SIZE (4 MB), MAX_FILE_SIZE (100 GB), VERSION, PORT, HOST, UPLOAD_DIR, REFRESH_INTERVAL_MS. By keeping settings in one file, changing behaviour never requires hunting through multiple modules.
`handler.py`	Python	HTTP router and API logic. The FiledropHandler class extends BaseHTTPRequestHandler and routes every GET and POST request to the correct private method. Contains all six API endpoints and the multipart form parser. The largest file but still under 200 lines.
`transfers.py`	Python	Thread-safe in-memory state manager. The TransferStore singleton tracks every active (in-progress) and completed transfer in Python dictionaries, protected by a threading.Lock. Exposes clean methods: init_transfer(), record_chunk(), complete_transfer(), snapshot_active(), snapshot_completed().
`utils.py`	Python	Standalone helper functions with no side effects. get_local_ip() opens a UDP socket to detect LAN IP. format_size() formats bytes into human-readable strings. get_mime() guesses MIME types. assemble_chunks() stitches chunk files into the final output. banner() prints the startup message.

File	Type	Responsibility & Explanation
`templates.py`	Python	Template loader and injector. Reads sender.html and receiver.html from the templates/ directory, replaces {{VERSION}}, {{CHUNK_SIZE}}, and {{REFRESH_INTERVAL}} placeholders with live values from Config, and returns the rendered HTML string.
`sender.html`	HTML	Sender page structure. Declares the drop zone, hidden file input, file list container, progress bar sections, status message divs, send button, and completed list. Contains no inline styles or scripts — those are in separate files.
`receiver.html`	HTML	Receiver page structure. Declares the status pulse indicator, transfers container, and empty state message. Extremely minimal — all dynamic rendering is done by receiver.js.
`common.css`	CSS	Shared visual styles. Defines two CSS variable themes (--theme-purple for sender, --theme-green for receiver), shared layout rules, card styles, status message colours, and the grid background texture. Both pages import this file first.
`sender.css`	CSS	Sender-specific styles. Drop zone dashed border, drag-over glow animation, file item cards, the shimmer animation on the progress bar fill, and the send button hover/active states.
`receiver.css`	CSS	Receiver-specific styles. Pulsing green dot animation, transfer card enter animation, progress bar, badge colours (amber for in-progress, green for complete), and the download button.
`common.js`	JS	Shared JavaScript utilities. Three pure functions: fmtSize(bytes) → human-readable string, fileIcon(filename) → emoji by extension, makeFileId() → random unique ID string. Loaded by both pages.
`sender.js`	JS	Chunked upload engine. Manages filesToSend array, wires up drag-drop events, orchestrates the three-phase upload protocol (init → chunks → complete), maintains rolling speed average for ETA calculation, and updates all progress UI elements.
`receiver.js`	JS	Polling and rendering loop. Fetches /api/status every 2 seconds, creates new transfer cards for unknown file_ids, updates progress bars and badges for existing cards, and appends a Download button when a transfer reaches complete status.

3.3 How the Modules Connect

The dependency chain flows strictly downward: server.py imports config.py, handler.py, and utils.py. handler.py imports config.py, transfers.py, utils.py, and templates.py. No circular dependencies exist. The static assets (CSS and JS) are served as plain files and have no import relationship with the Python modules — they are connected only at runtime via the browser loading them over HTTP.

```
server.py
   ├── config.py          (read settings)
```

```
    ├── handler.py         (handle requests)
    │    ├── config.py
    │    ├── transfers.py   (state management)
    │    ├── utils.py       (helpers)
    │    └── templates.py   (HTML loading)
    └── utils.py           (banner, IP detection)

 templates/ ←── templates.py reads these
 static/    ←── handler.py serves these
```

4. How It Works — Detailed Walkthrough

4.1 Server Startup Sequence

When the user runs `python server.py`, the following steps occur in order:

6. **Argument parsing:** argparse reads --port (default: 8765), --dir (default: ./received_files), and --host (default: 0.0.0.0) from the command line.
7. **Config injection:** The parsed values overwrite the defaults in the Config class. This means the rest of the application always reads from Config, not from argparse directly.
8. **Directory creation:** The upload directory is created if it does not exist, using Path.mkdir(parents=True, exist_ok=True). This is idempotent — safe to run multiple times.
9. **LAN IP detection:** utils.get_local_ip() opens a UDP socket, 'connects' to 8.8.8.8:80 (without actually sending data), and reads getsockname() to find the machine's outbound IP. This is the most reliable cross-platform method.
10. **Banner printing:** The startup banner displays the exact URLs for both the sender and receiver, as well as the ngrok command to use for internet transfers.
11. **Server start:** HTTPServer binds to the configured host and port and enters serve_forever(), which handles one request at a time in a loop.

> **Note on Threading**
>
> The default HTTPServer processes one request at a time. For normal use (one sender, one receiver), this is perfectly adequate. For concurrent transfers, replacing HTTPServer with ThreadingHTTPServer in server.py enables true parallel request handling with no other code changes needed.

4.2 The Three-Phase Upload Protocol

Every file transfer uses a strict three-phase protocol. This design ensures the server always knows exactly what to expect, makes error recovery straightforward, and keeps each phase independently testable.

Phase 1 — Initialisation

Before sending any data, `sender.js` generates a random `file_id` (a combination of a base-36 random number and a timestamp) and sends a POST to `/api/upload/init` with four fields:

- `file_id` — the unique identifier for this transfer session
- `filename` — the original file name, preserved exactly
- `size` — the total file size in bytes
- `total_chunks` — ceil(size / CHUNK_SIZE), so the server knows when it has received everything

The server registers the transfer in TransferStore, creates a temporary directory at `UPLOAD_DIR/<file_id>/`, and responds with `{"ok": true, "file_id": "..."}`.

Phase 2 — Chunk Upload Loop

sender.js loops from chunk index 0 to total_chunks - 1. For each iteration:

12. **Slice:** `file.slice(i * CHUNK_SIZE, min((i+1) * CHUNK_SIZE, file.size))` extracts one chunk as a Blob without copying it into memory.
13. **Form data:** The chunk is wrapped in a FormData object alongside `file_id` and `chunk_index`.
14. **POST:** Sent to `/api/upload/chunk` as `multipart/form-data`.
15. **Server writes:** handler.py parses the multipart body, extracts the binary chunk, and writes it to `chunk_XXXXXX` (zero-padded 6-digit index) in the temp directory.
16. **State update:** TransferStore.record_chunk() adds the chunk index to the received set and updates the byte count.
17. **Progress update:** The sender calculates instantaneous speed and a rolling 8-sample average. ETA = (total_bytes - bytes_sent) / avg_speed.

Phase 3 — Completion and Assembly

After the final chunk is acknowledged, sender.js sends a POST to `/api/upload/complete`. The server:

18. **Retrieves transfer info:** Reads total_chunks from the active store.
19. **Assembles the file:** Opens the output file for writing, then reads chunk_000000 through chunk_NNNNNN in order, appending each to the output. Each chunk file is deleted immediately after it is read.
20. **Updates state:** Moves the transfer from active to completed in TransferStore, recording the elapsed time.
21. **Logs the result:** Prints filename, total size, average speed, and elapsed seconds to the terminal.

4.3 Receiver Real-Time Updates

The receiver page does not use WebSockets or Server-Sent Events. Instead it uses simple polling: every 2,000 ms (configurable via Config.REFRESH_INTERVAL_MS), receiver.js fetches `GET /api/status`.

The response is a JSON object with two arrays: `active` (transfers currently receiving chunks) and `completed` (fully assembled files). For each entry:

- If the `file_id` is new, a transfer card is created in the DOM with an animated progress bar, a filename, total size, and a yellow 'Receiving…' badge.
- If the `file_id` already exists, the progress bar width, received bytes, and percentage text are updated in-place.
- When `status === 'complete'`, the badge turns green, the progress bar reaches 100%, and a Download button appears pointing to `/download/<file_id>`.

The polling approach was chosen over WebSockets because it requires no persistent connection, works through all proxies and tunnels, and the 2-second interval is imperceptible to the user while adding only negligible server load.

4.4 File Download

When the receiver clicks the Download button, the browser sends `GET /download/<file_id>` to the server. The handler:

22. Looks up the file_id in the completed store to get the filename.
23. Resolves the full path: `UPLOAD_DIR / file_id / filename`.
24. Sends HTTP 200 with `Content-Type` (guessed from the extension), `Content-Length` (exact file size), and `Content-Disposition: attachment`.
25. Streams the file in CHUNK_SIZE pieces so the server never holds the whole file in memory during download either.

4.5 Internet Access via ngrok

By default, the server listens on `0.0.0.0`, meaning it accepts connections from any IP. On a local network, this means any device on the same router can connect. For internet access, the user runs ngrok in a second terminal:

```
# Terminal 1 — start FileDrop
python server.py --port 8765

# Terminal 2 — create a public tunnel
ngrok http 8765

# ngrok will print something like:
# Forwarding  https://abc123.ngrok.io -> http://localhost:8765
```

ngrok creates an encrypted HTTPS tunnel from a public subdomain to the local port. The sender opens `https://abc123.ngrok.io/` and the receiver opens `https://abc123.ngrok.io/receive`. All traffic is encrypted in transit by ngrok's TLS.

Alternative tunnelling tools that work identically include Cloudflare Tunnel (`cloudflared tunnel --url http://localhost:8765`), Bore (`bore local 8765 --to bore.pub`), and LocalTunnel (`npx localtunnel --port 8765`).

5. System Flowchart

The diagram on the following page illustrates the complete data flow of the FileDrop system from the moment `python server.py` is executed until a file has been fully received and is available for download.

The chart is divided into three vertical lanes: the left lane shows actions taken by the sender's browser (JavaScript), the centre lane shows server-side processing (Python), and the right lane shows the receiver's browser and the state management store.

Key conventions used in the diagram:

- **Purple nodes:** Actions initiated by the sender browser (JavaScript in sender.js).
- **Green nodes:** Actions taken by the receiver browser (JavaScript in receiver.js) or response operations.
- **Teal nodes:** State management operations inside transfers.py and utils.py.
- **Grey nodes:** General server-side processing in handler.py.
- **Diamond:** A decision point — the chunk loop continues until all chunks are sent.
- **File Map (top right):** Shows all six Python modules and their roles for quick reference.

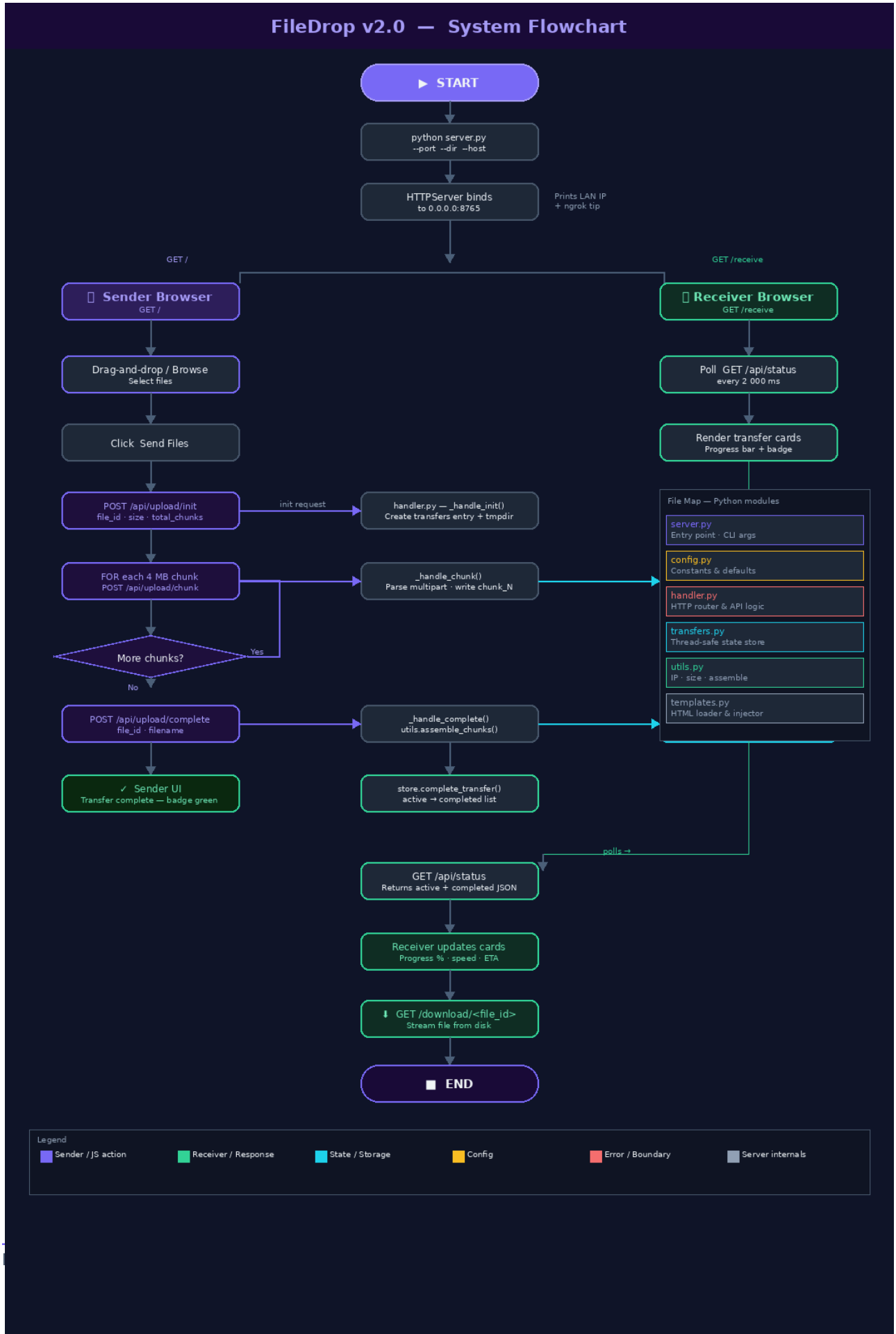
FileDrop v2.0 — System Flowchart
START
python server.py
--port --dir --host
HTTPServer binds
to 0.0.0.0:8765
Prints LAN IP
+ ngrok tip
GET /
GET /receive
Sender Browser
GET /
Receiver Browser
GET /receive
Drag-and-drop / Browse
Select files
Poll GET /api/status
every 2 000 ms
Click Send Files
Render transfer cards
Progress bar + badge
POST /api/upload/init
file_id · size · total_chunks
init request
handler.py — _handle_init()
Create transfers entry + tmpdir
File Map — Python modules
server.py
Entry point · CLI args
config.py
Constants & defaults
handler.py
HTTP router & API logic
transfers.py
Thread-safe state store
utils.py
IP · size · assemble
templates.py
HTML loader & injector
FOR each 4 MB chunk
POST /api/upload/chunk
_handle_chunk()
Parse multipart · write chunk_N
More chunks?
Yes
No
POST /api/upload/complete
file_id · filename
_handle_complete()
utils.assemble_chunks()
✓ Sender UI
Transfer complete — badge green
store.complete_transfer()
active → completed list
polls →
GET /api/status
Returns active + completed JSON
Receiver updates cards
Progress % · speed · ETA
GET /download/<file_id>
Stream file from disk
END
Legend
Sender / JS action
Receiver / Response
State / Storage
Config
Error / Boundary
Server internals

Figure 1: FileDrop v2.0 Complete System Flowchart

5.1 Flowchart Step-by-Step Walkthrough

The following table walks through each stage of the flowchart in sequence, explaining what happens at each node and why it is designed that way.

Step	Node / Action	Detailed Explanation
1	`START`	The Python process begins. server.py calls main(), which parses CLI arguments and sets Config values.
2	`python server.py`	The HTTPServer is instantiated and bound. FiledropHandler is registered as the request handler class.
3	`HTTPServer binds`	The server calls serve_forever(), entering a blocking loop that accepts incoming TCP connections.
4	`GET /  (Sender)`	A browser on the sender's machine opens the server URL. templates.py loads sender.html, injects config values, and returns it.
5	`GET /receive (Receiver)`	The receiver's browser opens the /receive URL. templates.py loads receiver.html and returns it.
6	`Select files`	The user drags files onto the drop zone or uses the file picker. sender.js adds each file to the filesToSend array and renders a file card.
7	`Poll /api/status`	receiver.js starts its 2-second polling loop, sending GET /api/status. At this point the response is empty arrays.
8	`Click Send Files`	The user clicks the button. sender.js sets transferring=true, disables the button, and iterates over filesToSend.
9	`POST /api/upload/init`	For the first file, sender.js generates a file_id and posts the transfer metadata. The server creates a temp directory and a state entry in TransferStore.
10	`FOR each chunk`	sender.js slices the file using File.slice() and enters the chunk loop. The loop counter i goes from 0 to total_chunks - 1.
11	`POST /api/upload/chunk`	Each chunk is sent as multipart/form-data. handler._handle_chunk() parses the boundary, extracts the binary data, writes it to chunk_XXXXXX, and calls store.record_chunk().
12	`record_chunk()`	TransferStore adds chunk_index to the received set and updates the received byte count. This update is immediately visible to the next /api/status poll.
13	`More chunks?`	Decision diamond. If i < total_chunks - 1, the loop continues. If all chunks are sent, execution continues to Phase 3.
14	`POST /api/upload/complete`	sender.js notifies the server that all chunks have been sent. handler._handle_complete() calls utils.assemble_chunks().

Step	Node / Action	Detailed Explanation
15	`assemble_chunks()`	utils.py opens the output file for writing and appends chunk_000000 through chunk_NNNNNN in order, deleting each chunk file as it goes.
16	`active → completed`	TransferStore.complete_transfer() removes the entry from _active and appends it to _completed. The elapsed time is recorded.
17	`Receiver polls`	On the next 2-second poll, GET /api/status returns the completed transfer. receiver.js sees the file_id with status='complete'.
18	`Download button`	receiver.js injects a Download anchor pointing to /download/<file_id>. When clicked, the server streams the assembled file to the browser.
19	`END`	The transfer is complete. The receiver has the file. The sender sees a green success message.

6. API Endpoint Reference

FileDrop exposes eight HTTP endpoints. Three are page routes that return HTML, three are upload API routes used by sender.js, one is a status polling route used by receiver.js, and one streams the assembled file for download. All endpoints are handled by the single FiledropHandler class in handler.py.

6.1 Page Routes

Method	Endpoint	Description
GET	`/ or /send`	Returns sender.html with {{VERSION}}, {{CHUNK_SIZE}} injected. The sender opens this page to begin a transfer.
GET	`/receive`	Returns receiver.html with {{REFRESH_INTERVAL}} injected. The receiver opens this page to watch incoming transfers.
GET	`/static/<path>`	Serves any file under static/css/ or static/js/ with a 1-hour Cache-Control header. This is how common.css, sender.js, etc. are delivered to the browser.

6.2 Transfer API Routes

These three routes implement the upload protocol described in Section 4.2. They are called exclusively by sender.js.

Method	Endpoint	Request / Response
POST	`/api/upload/init`	Request body (JSON): { file_id, filename, size, total_chunks }. Creates temp directory at UPLOAD_DIR/<file_id>/ and registers the transfer in TransferStore. Response: { ok: true, file_id }.
POST	`/api/upload/chunk`	Request body (multipart/form-data): fields file_id (string), chunk_index (integer), chunk (binary). Writes binary data to chunk_XXXXXX file. Response: { ok: true }.
POST	`/api/upload/complete`	Request body (JSON): { file_id, filename }. Calls assemble_chunks(), moves transfer to completed, logs timing. Response: { ok: true, file_id }.

6.3 Status and Download Routes

Method	Endpoint	Description
GET	`/api/status`	Returns JSON: { active: [{file_id, filename, size, received, status:'active'},...], completed: [{..., status:'complete'},...] }. Called every REFRESH_INTERVAL_MS by receiver.js.
GET	`/download/<file_id>`	Streams the assembled file from UPLOAD_DIR/<file_id>/<filename> with Content-Disposition: attachment. Uses chunked reads (CHUNK_SIZE) to avoid loading the full file into memory.

7. Performance and Scalability

7.1 Why Chunking Enables Large File Transfers

Without chunking, a file upload works by loading the entire file into browser memory as a single request body. For a 10 GB file on a machine with 8 GB of RAM, this is immediately fatal. Even if the machine had enough RAM, a single 10 GB HTTP request body takes tens of minutes on a typical connection, during which a single TCP timeout or brief network interruption would restart the transfer from zero.

Chunking solves both problems:

- **Memory:** File.slice() in the browser does not copy data — it creates a view into the original file. The 4 MB chunk is held in memory only for the duration of one fetch() call, then garbage collected. Total memory usage at any moment: ~4 MB regardless of file size.
- **Resilience:** A network error on chunk 847 of 1000 means only chunk 847 needs to be retried. The previous 847 chunks are already safe on disk. (The current implementation retries the whole transfer on error; resume support is listed as a future enhancement.)
- **Server memory:** Each chunk is written to disk immediately upon receipt in _handle_chunk(). The server holds at most one 4 MB chunk in memory per concurrent transfer at any time.

7.2 Transfer Speed Estimates

The table below shows estimated transfer times under ideal conditions (no other network traffic, wired connections where applicable). Real-world speeds will vary based on CPU load, disk I/O, and network congestion.

Network	Bandwidth	1 GB	10 GB	100 GB
Gigabit Ethernet (wired LAN)	~100 MB/s	~10 s	~1.5 min	~17 min
Wi-Fi 6 (5 GHz, close range)	~60 MB/s	~17 s	~2.8 min	~28 min
Wi-Fi 5 (5 GHz, typical)	~30–40 MB/s	~28 s	~4.5 min	~45 min
Wi-Fi 4 (2.4 GHz)	~8–12 MB/s	~100 s	~16 min	~2.8 hr
Internet (500 Mbps fibre)	~55 MB/s	~18 s	~3 min	~30 min
Internet (100 Mbps)	~11 MB/s	~92 s	~15 min	~2.5 hr
Internet (50 Mbps upload)	~5.5 MB/s	~3 min	~30 min	~5 hr

7.3 Concurrency and Thread Safety

Every read and write to the `active_transfers` and `completed_transfers` data structures in TransferStore is wrapped in a `threading.Lock` context manager. This means that even if two senders upload chunks simultaneously, there are no race conditions in state updates.

The lock is held only for the duration of the dictionary update, not during disk I/O. Chunk files are written outside the lock, which means disk write operations never block status polling. This design keeps the server responsive even under concurrent load.

7.4 Disk Space Considerations

During a transfer, FileDrop temporarily uses approximately 2× the file size on disk: once for the incoming chunk files (`chunk_000000` through `chunk_NNNNNN`) and once for the assembled output file. After assembly, the chunk files are deleted immediately, returning to 1× disk usage. Users should ensure the receiving drive has at least 2× the size of the largest expected transfer as free space.

8. Configuration Reference

All tunable parameters are defined as class attributes on the `Config` class in `config.py`. Changing a value here affects the entire application — Python server logic, the startup banner, and the HTML/JavaScript injected into browser pages (via templates.py). There is intentionally no separate JSON config file: keeping settings in Python makes them type-safe and IDE-navigable.

Setting	Default	Type	What It Controls
`CHUNK_SIZE`	4×1024^2 (4 MB)	int	Size of each upload piece in bytes. Increase to 8 MB or 16 MB for fast wired LAN; decrease to 1 MB for slow or unreliable connections.
`MAX_FILE_SIZE`	100×1024^3 (100 GB)	int	Soft cap enforced by sender.js before upload begins. The server has no hard limit beyond available disk space.
`REFRESH_INTERVAL_MS`	2000	int	How often receiver.js polls /api/status in milliseconds. Lower = more responsive UI, higher = less server load.
`PORT`	8765	int	HTTP port. Override at runtime with --port. Choose a port above 1024 to avoid needing admin/root privileges.
`HOST`	0.0.0.0	str	Bind address. 0.0.0.0 accepts connections from any IP. Set to 127.0.0.1 for localhost-only (no remote access).
`UPLOAD_DIR`	`./received_files`	Path	Root directory for all file storage. Chunk files and assembled output files are stored under <UPLOAD_DIR>/<file_id>/.
`VERSION`	2.0.0	str	Version string injected into the sender UI and the startup banner. Update when releasing new versions.

8.1 Command-Line Overrides

The three most commonly changed settings (PORT, HOST, UPLOAD_DIR) can be overridden at launch without editing any file:

```
# Use a different port
python server.py --port 9000

# Save files to a specific external drive
python server.py --dir /media/usb-drive/transfers
```

```
# Listen on localhost only (no remote access)
python server.py --host 127.0.0.1

# All options combined
python server.py --port 9000 --dir /data/filedrop --host 0.0.0.0
```

9. Security Considerations

FileDrop is designed for use in trusted environments (a home network, an office LAN, or a temporary ngrok tunnel shared with a known recipient). It does not include authentication, encryption, or access control out of the box. Users operating in untrusted environments should be aware of the following:

9.1 No Authentication

Anyone who can reach the server URL can upload files to the receiver's machine and download completed files. On a home or office LAN this is generally acceptable. Over a public ngrok tunnel, the URL itself acts as an access token (it is a random subdomain), but this is security by obscurity — not a robust access control mechanism.

> **Recommendation**
>
> For internet transfers to untrusted parties, add a shared password to the URL or implement HTTP Basic Authentication in handler.py. This is listed as a high-priority enhancement in Section 10.

9.2 No Transport Encryption (LAN)

The LAN server runs over plain HTTP. Traffic on a local network is not encrypted. For most home and office use cases this is acceptable, but for sensitive files on a shared corporate network, consider running the server behind a reverse proxy with TLS (e.g., nginx with a self-signed certificate).

Note: ngrok automatically provides HTTPS for internet transfers, so the internet-access mode is encrypted end-to-end.

9.3 File Name Handling

File names are taken directly from the browser's File object and used as the output file name on disk. The server does not sanitise file names. In the current implementation, path traversal attacks (e.g., a file named `../../etc/passwd`) are mitigated by the fact that the output path is constructed as `UPLOAD_DIR / file_id / filename`, where `file_id` is a random string that acts as a sandboxing namespace. However, explicit path sanitisation should be added in a production deployment.

10. Future Enhancements

The following enhancements are planned for future versions. They are listed in priority order based on impact and implementation complexity.

Enhancement	Priority	Description
Password Protection	🔴 High	Add a shared secret key to the URL (?key=xxxx) or implement HTTP Basic Auth in handler.py. Essential for public-internet transfers to prevent unauthorized access.
SHA-256 Checksum Verification	🔴 High	Compute a SHA-256 hash of the file in sender.js using the Web Crypto API and include it in the /complete payload. Server verifies the hash of the assembled file. Guarantees byte-perfect delivery.
Resume Interrupted Transfers	🔴 High	Add a GET /api/upload/status/<file_id> endpoint that returns the set of already-received chunk indices. sender.js can query this on page load and skip already-uploaded chunks.
Parallel Chunk Uploads	🟡 Medium	Upload 4–6 chunks concurrently using Promise.all() instead of awaiting each sequentially. Can increase LAN throughput by 3–4× by saturating network bandwidth rather than waiting for round-trip acknowledgements.
QR Code for Sender URL	🟡 Medium	Generate a QR code image on the receiver page using the qrcode library, encoding the sender URL. Allows mobile devices to connect by scanning the screen.
Transfer History Persistence	🟡 Medium	Write completed transfer entries to a JSON log file on disk in complete_transfer(). Reload it on startup. Transfer history then survives server restarts.
HTTPS / TLS Support	🟡 Medium	Wrap HTTPServer with ssl.wrap_socket() using a self-signed certificate. Required for the browser's File API to work reliably on HTTPS-only networks.
Multi-Receiver Broadcasting	🟢 Low	Allow one sender to deliver the same file to multiple waiting receivers simultaneously. Each receiver would download directly from the server.
Dark / Light Theme Toggle	🟢 Low	Add a toggle button to both UIs that switches between the current dark theme and a light variant using CSS custom property swapping.
Transfer Speed Throttling	🟢 Low	Add a --max-speed option to config.py that limits upload throughput, useful when sharing a network with other users.

11. Quick Start Guide

This section is a self-contained reference for anyone who wants to run FileDrop without reading the full report.

11.1 Requirements

- **Python 3.6 or later** (check with `python --version`). No pip packages needed.
- **ngrok** (optional, for internet transfers only). Download from https://ngrok.com/download.
- **Any modern browser** (Chrome, Firefox, Safari, Edge) on both sender and receiver machines.

11.2 LAN Transfer (Step by Step)

26. **On the RECEIVER's machine:** open a terminal in the filedrop/ folder and run `python server.py`. The terminal will print the URLs.
27. **Share the Sender URL** (e.g., `http://192.168.1.42:8765/`) with the person sending the file.
28. **Open the Receiver URL** (e.g., `http://192.168.1.42:8765/receive`) in a browser on the receiver's machine.
29. **The sender** opens the Sender URL in their browser, drags files onto the drop zone, and clicks Send Files.
30. **Watch the progress** on both screens. When the transfer is complete, the receiver clicks Download.

11.3 Internet Transfer (Step by Step)

31. **On the RECEIVER's machine:** run `python server.py` in one terminal.
32. **In a second terminal:** run `ngrok http 8765`. Copy the `https://xxxx.ngrok.io` URL.
33. **Share** `https://xxxx.ngrok.io/` (sender) and `https://xxxx.ngrok.io/receive` (receiver) with the appropriate parties.
34. **Continue as above** from Step 4 of the LAN guide.

> **Pro Tip**
>
> For the fastest possible LAN transfer, connect both machines to the same router with Ethernet cables rather than Wi-Fi. This can increase speed from ~30 MB/s (Wi-Fi 5) to ~100 MB/s (Gigabit Ethernet) — a 3× improvement for a 100 GB file.

12. Conclusion

FileDrop v2.0 demonstrates that a production-quality large-file transfer system can be built with Python's standard library alone — no frameworks, no databases, no external packages. The total codebase spans 14 files and approximately 900 lines of code, yet it reliably handles transfers that would defeat every standard consumer file-sharing tool.

The architecture succeeds because of three disciplined design decisions. First, chunking: by never loading more than 4 MB of any file into memory at once, the system can handle arbitrarily large files on machines with ordinary RAM. Second, separation of concerns: by giving each language (Python, HTML, CSS, JavaScript) and each responsibility (routing, state, utilities, templates) its own file, the codebase remains understandable and maintainable as it grows. Third, stateless polling: by having the receiver poll a simple JSON endpoint rather than maintaining a WebSocket, the system works through every proxy, tunnel, and firewall without special configuration.

The three high-priority enhancements — password protection, checksum verification, and transfer resume — would bring FileDrop to production readiness for deployment in untrusted environments. The medium-priority enhancements would significantly improve performance and usability. All of them build naturally on the existing architecture without requiring structural changes.

FileDrop is ready for immediate use on local networks today. With ngrok, it works globally. It is the simplest possible answer to a problem that affects anyone who has ever tried to move a large file between two computers.